Ders 3 Genişletilmiş Notu: Sparse String, Baudot, Önek Sıkıştırma ve Run Length Encoding (RLE)

Ders 3 Genişletilmiş Notu: Sparse String, Baudot, Önek Sıkıştırma ve Run Length Encoding (RLE)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ad Hoc Sıkıştırma Yöntemleri
- 2. Sparse String Sıkıştırma
- 3. Baudot Kodlaması
- 4. Önek Sıkıştırma (Front Compression)
- 5. Run Length Encoding (RLE): Temel Fikir
- 6. RLE'de Run ile Veriyi Ayırt Etme: Escape Kodu
- 7. RLE'nin Temel Problemleri ve Versiyonları
- 8. MNP5 ve RLE ile Karşılaştırılması
- 9. Görüntüde RLE: Piksel, Renk Derinliği ve Nicemleme
- 10. Bitmap'in Taranması ve Siyah-Beyaz Resimde RLE
- 11. Renkli Resim Örnekleri ve Run Uzunluğunun Bit Sayısı
- 12. Grayscale ve True Color Resimlerde RLE Versiyonları
- 13. Tarama Yönünün Seçimi ve Zigzag Okuma
- 14. Kayıplı RLE ve RLE'nin Özeti
- Örnek Sorular

1. Ad Hoc Sıkıştırma Yöntemleri

Hoca derse geçen haftanın yarım kalan konusundan devam ederek başladı. Geçen hafta sparse string sıkıştırmanın slaytında çok fazla sıfır birbirine girdiği için örnek yarım kalmıştı; slayt bu hafta düzeltilmiş hâliyle anlatıldı. Hoca bu dersi daha önce tahtada renkli tebeşirlerle adım adım anlattığını, slaytlarda da farklı renkli kalemlerle göstermeye çalışacağını, bu yüzden dikkatle izlenmesi gerektiğini söyledi.

Bu bölümdeki yöntemler ad hoc sıkıştırma yöntemleridir. Hoca "ad hoc" kelimesinin ağlarda (ad hoc network) da geçtiğini hatırlattı. Burada ad hoc yöntem, bir kişiye ya da bir probleme özgü olarak ortaya atılmış yöntem demektir: elimizde kendi problemimize ait bir veri vardır ve aynı probleme sahip başka biri olmayabilir. Bu yöntemler genel amaçlı değildir, iddiaları da çok iyi sıkıştırma yapmak değildir. Günümüzde tercih edilen yöntemler değildir, ama sıkıştırmanın ilk girişlerinden bugüne nereden nereye gelindiğini görmek için incelenir.

2. Sparse String Sıkıştırma

2.1 Problem

Sparse veri, içinde bol sıfır bulunan veridir. Burada elimizde bir doküman ya da metin vardır ve metinde kelimeler arasında birden fazla boşluk bulunabilir: iki, üç, dört boşluk. Akla şu soru gelir: Neden bütün boşlukları bir ön işlemle tek boşluğa indirmiyoruz? Hocanın cevabı: İndire-

bilirsin, ama o zaman orijinal veriye birebir dönemezsin. Amaç, boşluk sayılarını kaybetmeden, kayıpsız sıkıştırmaktır.

Yöntemde her karakterin yerine 0 konduğu için ortaya çok sıfırlı bir dizi çıkar; adı da buradan gelir.

2.2 Örnek metin ve L1 seviyesi

Örnek metin (ekrandan, 00:10:00; _ boşluğu gösteriyor):

```
Bu__dersin__adı___veri_sıkıştırma___ve_pazartesi_günü__yapılıyor
```

Slaytta her boşluk farklı bir renkle gösterilmişti; kelimeler arasında 2, 2, 3, 1, 3, 1, 1, 2 boşluk vardır.

Algoritmada L1, L2, L3 … seviyeleri (level) vardır. Seviye sayısı metne bağlıdır; n tane olabilir ama bir yerde durur. Bir cümle yerine bir paragraf da alınabilir ya da "her beş cümlede bir algoritmayı yeniden çalıştırırım" denebilir. Bu örnekte bir cümle alınmıştır.

Birinci seviyede her karakter yerine 0, her boşluk yerine 1 yazılır ve bitler 4'lük bloklara ayrılır. "Bu" iki karakterdir: `00`; ardından iki boşluk: `11`; ilk blok `0011` olur. "dersin" 6 karakterdir, 6 sıfır gelir: 4'ü ikinci blokta, 2'si üçüncü bloğun başında. Sonra iki boşluk `11`, "adı" için üç sıfır, üç boşluk için üç bir gelir ve bu böyle sürer. Sonuç (ekrandan, 00:10:00; python ile doğrulandı):

```
L1 = 0011|0000|0011|0001|1100|0010|0000|0000|0111|0010|0000|0000|1000|0110|0000|0000
```

2.3 L2 ve L3 seviyeleri

İkinci seviyede her 4'lü bloğa bakılır: blok sıfırdan farklıysa yerine 1, tamamen sıfırsa 0 yazılır. L1'in 16 bloğundan 16 bit çıkar ve bunlar da 4'lü bloklara ayrılır:

```
L2 = 1011|1100|1100|1100
```

Aynı işlem L2'ye uygulanır. L2'nin dört bloğunun hepsi sıfırdan farklı olduğu için:

```
L3 = 1111
```

Artık L4 yazıp tek bir 1 koymanın anlamı yoktur. Kural şudur: kısaltılabilecek bir durum kalmayıncaya kadar seviye eklenir.

2.4 Sıkıştırma: sıfır blokların silinmesi

Seviyeleri çıkarmak tek başına sıkıştırma değildir. Asıl sıkıştırma için L1'deki sıfır bloklar atılır (ekrandan, 00:20:00):

```
L1 = 0011|0011|0001|1100|0010|0111|0010|1000|0110
L2 = 1011|1100|1100|1100
L3 = 1111
```

L2'de sıfır blok yoktur, aynen alınır; L3 de aynen alınır. Hoca her bloğu farklı renkle göstermişti, böylece açarken çizilecek ağaçta hangi bloğun nereden geldiği kolay izlenir. Çıktı akışına (output stream) sırasıyla L1, L2, L3 bit dizileri ve ardından metin boşluksuz olarak yazılır.

2.5 Başlık (header)

Bir soru doğar: L1, L2, L3 arka arkaya yazıldıysa L1 nerede biter, L2 nerede başlar? Bunun için başa birkaç bayt eklenir. L1'de 9 blok kaldı, $9 \cdot 4 = 36$ bit; L2'de 4 blok, $4 \cdot 4 = 16$ bit; L3 4 bittir. Seviye sayısını bilmek için de en başa bir bayt daha konur:

```
3 | 36 | 16 | 4 | L1 (36 bit) L2 (16 bit) L3 (4 bit) | budersinadıverisıkıştırmavepazartesigün
```

Açan taraf ilk baytta 3'ü görünce 3 seviye olduğunu anlar; sonraki baytlardan L1 için 36, L2 için 16, L3 için 4 bit okuyacağını öğrenir. Ardından sıkıştırılmış metin gelir.

Ek bilgi (derste anlatılmadı): Bu örnekte metin 64 karakterdir (49 karakter + 15 boşluk). Çıktı 4 baytlık başlık, $36 + 16 + 4 = 56$ bit (7 bayt) seviye bilgisi ve 49 karakterden oluşur, yani yaklaşık 60 bayttır (ajan hesabı). Kazanç küçüktür; yöntem uzun ve çok boşluklu metinlerde anlam kazanır.

2.6 Açma

Hoca burayı dikkatle izlemeyi önerdi. Sparse string simetrik bir algoritmadır: sıkıştırırken ne yapıldıysa açarken tam tersi yapılır. En son L3 oluşturulduğu için açma L3'ten başlar (ekrandan, 00:20:00):

```
                    1111
   1011         1100         1100         1100
0011 0000 0011 0001 | 1100 0010 0000 0000 | 0111 0010 0000 0000 | 1000 0110 0000 0000
```

- L3'ün ilk biti 1'dir: L2'deki ilk dörtlü (`1011`) alınır. İkinci bit 1: ikinci dörtlü alınır. Üçüncü ve dördüncü bitler de 1 olduğu için L2'nin dört bloğunun hepsi alınır. Bu örnekte tesadüfen hiç 0 çıkmamıştır.
- L2'den L1'e dönülür. `1011`'in ilk biti 1: L1'den ilk blok (`0011`) alınır. İkinci bit 0: sıkıştırırken sıfır bloklar atıldığı için buraya `0000` yazılır. Sonraki iki 1 için L1'in ikinci ve üçüncü blokları alınır. Diğer L2 bloklarında da 1 görülünce L1'den sıradaki blok alınır, 0 görülünce `0000` yazılır.

Böylece 64 bitlik orijinal L1 geri elde edilir. Son adımda L1'de her 0 için sıkıştırılmış metinden sıradaki karakter, her 1 için bir boşluk yazılır: `00` → "Bu", `11` → iki boşluk, altı sıfır → "dersin", `11` → iki boşluk, üç sıfır → "adı", üç bir → üç boşluk ve böyle devam eder. Metin aynı boşluklarla, hiçbir eksik olmadan geri gelir.

Ders sonrası not: Slayttaki boşluksuz metin `budersiadıverisıkıştırmavepazartesigünüyapılıyor` olarak yazılmış, "dersin" kelimesinin `n`'si eksik. Hoca açarken altı karakter alması gerektiğini sayınca bunu fark etti (00:20:46). Doğrusu `budersinadıverisıkıştırmavepazartesigünüyapılıyor`'dur.

Soru: Sıfır olan blokları silince karakter bloklarını, yani karakter bilgisini kaybetmiyor muyuz?
Cevap: Hayır. Karakterler kaybolmaz, çünkü hepsi çıktı akışında arka arkaya yazılıdır. Sıfırlar yalnız karakterlerin nereye geleceği bilgisini verir. L1'den sıkıştırılmış L1'e geçerken yalnız sıfırdan farklı bloklar alınır, ama L2 oluşturulurken sıfır blokların yeri korunmuştur (L2'deki 0 bitleri). Açarken bu bilgiden yararlanılır.

3. Baudot Kodlaması

Baudot kodlaması, adını bulan kişinin soyadından alır. Hoca çok basit ama zekice bir kodlama olduğunu söyledi. Her sembole 5 bitlik bir kod verilmiştir (ekrandan, 00:30:00).

Harf	Kod	Figür	Harf	Kod	Figür
A	`10000`	1	Q	`10111`	/
B	`00110`	8	R	`00111`	-
C	`10110`	9	S	`00101`	SP
D	`11110`	0	T	`10101`	na
E	`01000`	2	U	`10100`	4
F	`01110`	na	V	`11101`	’
G	`01010`	7	W	`01101`	?
H	`11010`	+	X	`01001`	,
I	`01100`	na	Y	`00100`	3
J	`10010`	6	Z	`11001`	:
K	`10011`	(	LS	`00001`	LS
L	`11011`	—	FS	`00010`	FS
M	`01011`	)	CR	`11000`	CR
N	`01111`	na	LF	`10001`	LF
O	`11100`	5	ER	`00011`	ER
P	`11111`	%	na	`00000`	na

Tabloda A'dan Z'ye büyük harfler ve figürler (0-9 rakamları, parantez, artı, eksi, virgül, boşluk gibi işaretler) vardır. Özel kodlar sağ altta toplanmıştır ve harf ile figür sütununda aynıdır: LS (Letter Shift), FS (Figure Shift), CR (Carriage Return), LF (Line Feed), ER (Error), na (Not Assigned, atanmamış), SP (Space).

> **Ders sonrası not:** Hoca CR için "bir satır atladık", LF için "satır başı" dedi. Doğrusu tersidir: Carriage Return satır başına dönmek, Line Feed bir satır aşağı geçmektir.

3.1 Harf ve rakamın karışması

Hoca `AB46` dizisini kodladı (ekrandan, 00:30:00): A `10000`, B `00110`, 4 (U'nun figürü) `10100`, 6 (J'nin figürü) `10010`. Bu kodlar karşı tarafa gönderildiğinde açıcı `10000`'ı tabloda arar ve iki karşılık bulur: A mı, 1 mi? Nasıl anlayacağını bilemez.

Çözüm LS ve FS kodlarıdır. Harf okunacaksa başa LS (`00001`) yazılır; anlamı "bundan sonraki bütün kodları harf olarak oku"dur. Rakam başlayacaksa FS (`00010`) yazılır; "bundan sonrakileri figür olarak oku" demektir:

```
LS     A      B      FS     4      6
00001  10000  00110  00010  10100  10010
```

3.2 Atanmamış kodlar

Tablodaki na kodlarına hiçbir şey atanmamıştır; kullanıcı istediği sembolü verebilir. Hoca örneğin birine "Caps" (büyük harfe geç) anlamı verilebileceğini, iki tane art arda konursa "küçük harfe geç" anlamının verilebileceğini söyledi. Atanmamış kodlar bu tür konularda esneklik sağlar.

Hoca bu yöntemin çok iyi bir sıkıştırma yaptığı iddiasında olmadıklarını, ihtiyacı olanın kullanabileceği bir ad hoc yöntem olduğunu belirtti.

4. Önek Sıkıştırma (Front Compression)

Hoca bu yöntemi geçmişte ödev olarak verdiğini söyledi. Elimizde bir sözlük ya da daha doğrusu bir yazım kılavuzu olsun (kelimelerin anlamları yok, yalnız kelimeler). Kelimeler alfabetik sıradadır: önce birinci harfe, sonra ikinci, üçüncü harfe göre sıralanmıştır. Bu kılavuz nasıl sıkıştırılır?

Fikir: Her kelime, bir üst satırdaki kelimeyle soldan itibaren arka arkaya örtüşen karakter sayısı ve ardından kalan kısım olarak yazılır (ekrandan, 00:32:30):

Kelime	Kod	Kelime	Kod
a	a	abash	3sh
aardvark	1ardvark	abated	3ted
aback	1back	abate	3te
abaft	3ft	abbot	2bot
abandon	3ndon	abbey	3ey
abandoning	3ndoning	abbreviating	3reviating
abasement	3sement	abbreviate	9e
abandonment	3ndonment	abbreviation	9ion

- İlk kelime `a` aynen yazılır.
- `aardvark` ile üstteki `a` arasında bir karakter ortaktır: `1ardvark`. Buradaki 1, "üst satırdaki kelimenin soldan birinci karakterini al" demektir.
- `aback` için de üstteki `aardvark` ile yalnız `a` ortaktır: `1back`.
- `abaft`: üstteki `aback` ile soldan `aba` ortaktır, 3 yazılır, ardından `ft` gelir. Çözerken üstteki kelimenin ilk üç karakteri alınır; o kelimenin de kendi başı yukarıdaki kelimeye bağlıdır.
- `abated` 6 karakterdir ve `3ted` ile 4 karakterle yazılır.
- `abbreviate` 10 karakterdir; üstteki `abbreviating` ile ilk 9 karakteri aynıdır: `9e`, yani 10 bayt 2 baytla yazılır.

Sözlüğün ilk başlarında kazanç azdır; ortak kısmı uzun kelimeler arka arkaya geldiğinde kazanç artar. Hoca bu performansın %70-80 gibi yüksek oranlar olmadığını söyledi. Ortaklık yoksa (hocanın örneği: "ay" ardından "bay") kelime olduğu gibi yazılır. Çözme sırasında art arda yukarıya gitmek gerekebileceği için hızlı çözmek amacıyla bağlı liste (linked list) mantığı kullanılabilir.

Hocanın vurgusu, bu yöntemlerin bizim bile bakınca görebileceğimiz kadar basit olmasıdır: Bir yazım kılavuzunu sıkıştırın denseydi, algoritma bilen biri stringlerde örtüşen kısımlar olup olmadığına bakardı.

> **Ders sonrası not:** Slayttaki tabloda iki satır bir üstteki kelimeyle uyuşmuyor (python ile kontrol edildi). `abandoning`, üstündeki `abandon` ile 7 karakter ortak olduğu için `7ing` olmalıdır (slaytta `3ndoning`). `abate`, üstündeki `abated`'in ilk 5 karakteri olduğu için yalnız `5` yazılabilir (slaytta `3te`). Ayrıca alfabetik sırada `abandonment`, `abasement`'tan önce gelir.

5. Run Length Encoding (RLE): Temel Fikir

5.1 Tanım

Slaytın başlığı "Run Length Compression"dı; hoca bunu "Run Length Coding / Encoding" olarak düzeltti ve kısaltmasının RLE olduğunu söyledi (ekrandan, 00:40:00). Slayttaki tanımlar:

- Bir input stream içinde bir `d` verisi `n` defa arka arkaya geliyorsa bunu `nd` çifti ile gösterebiliriz.
- `n` değerine run denir; yöntemin adı buradan gelir.
- Hem metin hem görüntü sıkıştırmada kullanılır.
- Kayıplı ve kayıpsız versiyonları vardır. Kayıpsız bir yöntemdir, ama bazı varyasyonları kayıplıdır.

Örnek (ekrandan, 00:41:40): Elde `111` varsa `3 1` yazılır; 3 run, 1 verinin kendisidir.

5.2 Nerede başarılıdır?

RLE metinde de görüntüde de kullanılabilir, ama metinde başarılı olmaz. Görüntüde başarılıdır; siyah-beyaz görüntülerde çok daha başarılıdır. Buradaki siyah-beyaz, bir siyah bir beyaz piksel diye değişen bir resim değil, büyük bir grup beyaz ve büyük bir grup siyahtan oluşan resimdir.

Hoca RLE'yi tek başına kullanmanın çok mantıklı olmadığını, başka yöntemlerle birlikte kullanıldığını söyledi: JPEG'in ara adımlarından birinde RLE vardır; WinRAR ve ZIP içinde de geçtiğini söyledi. Resim ya da metin üzerinde belirli bir ön işlem yapıldıktan sonra RLE uygulanınca performans artar. Tek başına kullanıldığında ancak özel durumlarda (dokümanda çok fazla boşluk olması, resmin siyah-beyaz olması) iyi sonuç verir.

> **Ders sonrası not:** ZIP'in standart sıkıştırma yöntemi Deflate (LZ77 + Huffman) büyük olasılıkla ayrı bir RLE adımı içermez; ayrı bir RLE adımı bzip2 gibi araçlarda bulunur.

5.3 Başarılı ve başarısız iki örnek

Birinci örnek (ekrandan, 00:41:40 ve 00:43:40):

```
Uncompressed: aaaaabbbbbbbbbbbbccccddddddddddeeeeeeeeee
Compressed:   5a12b4c9d10e
```

5 tane `a`, 12 tane `b`, 4 tane `c`, 9 tane `d`, 10 tane `e` vardır; girdinin boyu $5+12+4+9+10=40$ bayttır. Çıktıda 12 ve 10 iki rakam gibi görünse de her run tek bayta yazılır (bir bayt 255'e kadar sayı tutar). Çıktı $5 \times 2 = 10$ bayttır: 40 bayt 10 bayta iner.

İkinci örnek (ekrandan, 00:43:40):

```
Uncompressed: aabccdeefghijjjklmnopqrrstttuvvwwxyyyz      (38 bayt)
Compressed:   2a1b2c1d2e1f1g1i3j1k1l1m1n1o1p1q2r1s3t1u2v2w1x3y1z
```

Bu kez çıktı girdiden büyüktür; slaytta “negative compression” yazar. Sebebi basittir: İki `a` zaten 2 bayt tutar, `2a` da 2 bayttır, kazanç yoktur. Tek `b` 1 bayttır, `1b` 2 bayt olur. `d`, `f`, `g`, `h`, `i` gibi tek geçen karakterlerin her biri 1 bayttan 2 bayta çıkar.

> **Ders sonrası not:** Slayttaki sıkıştırılmış dizide `1g`'den sonra `1h` atlanmış. 38 karakterlik girdinin doğru RLE çıktısı `2a1b2c1d2e1f1g1h1i3j1k1l1m1n1o1p1q2r1s3t1u2v2w1x3y1z` olup 52 karakterdir (python ile kontrol edildi).

Hoca buradan genel sonucu çıkardı: Dosyada arka arkaya tekrar eden karakterler (metinde) ya da piksel değerleri (görüntüde) ne kadar uzunsa RLE'nin performansı o kadar iyidir.

5.4 Metin mi, görüntü mü?

Metinde aynı karakterin arka arkaya gelme olasılığı düşüktür. Hoca Türkçede “saat” ve “yolluk” gibi örnekler verdi; bunlar bile ikili tekrarlardır. Yalnız metin kodlanırsa RLE hiçbir zaman başarılı olmaz, negatif sıkıştırma elde edilir. Ama bir A4 sayfasında yalnız birkaç satır yazı olup geri kalanı boşluksa RLE başarılı olur.

Görüntüde ise bir elma düşünülürse elmanın kırmızı pikselleri arka arkaya gelir. Fakat resim true color ise, yani 2^{24} farklı renk olabiliyorsa, aynı piksellerin arka arkaya gelmesi neredeyse imkânsızdır. En iyi sonuç siyah-beyaz resimlerde alınır; orada siyah ve beyaz piksellerin sayısı arka arkaya çok fazla gelir.

Soru: JPEG'te RLE kullanılıyor dendi, ama JPEG dosyaları siyah-beyaz değil renkli resimler. RLE nasıl işe yarıyor? **Cevap:** JPEG'in ön işlemlerinde DCT (Discrete Cosine Transform) gibi dönüşümler yapılır. Sayısal değerler belirli matrislerden geçirilerek birbirine yakın değerler hâline getirilir; böylece RLE'nin işine yarayan tekrarlar ortaya çıkar.

6. RLE'de Run ile Veriyi Ayırt Etme: Escape Kodu

6.1 Problem

Açıcı bir sayının run mı, yoksa metnin kendi içindeki bir sayı mı olduğunu nereden bilecek? Örneğin metinde “Cumhuriyet 1923 yılında ilan edildi” yazıyorsa 1923 run değeri midir? Hocanın örneği (ekrandan, 00:50:00):

```
Uncompressed: 2. all is too well
Compressed:   2. a2l is t2o we2l
```

`2`, `.`, `a` olduğu gibi alınır; iki `l` için `2l`, `is` ve `t` olduğu gibi, iki `o` için `2o`, iki `l` için yine `2l` yazılır. Açıcı baştaki `2`'yi run sanıp iki nokta yazabilir. Orijinale dönülemiyorsa bu bir sıkıştırma yöntemi değildir.

Hoca ayrıca bu örnekte sıkıştırma olup olmadığını sordu: Olmadı. İki `l` yerine 2 bayt, iki `o` yerine 2 bayt konmuştur; zaten bu kadar küçük bir metinden sıkıştırma beklenmez.

6.2 Çözüm: escape sembolü

Hoca öğrencilere çözümü sordu. Bir öğrenci, sıkıştırılan karakterlerin önüne bir sembol konmasını önerdi. Hocanın çözümü de buydu: özel bir escape kodu kullanılır; derste `@` seçildi. Açıcı escape sembolünü görünce sonraki sayının run olduğunu anlar:

```
2. a@2l is t@2o we@2l
```

Baştaki `2` metnin kendisi olduğu için önüne escape konmaz. Açıcı `2`, `.`, `a` yazar; escape'i görünce sonraki sayının run olduğunu anlar ve iki `l` yazar; `is`, `t` yazar; yine escape görünce iki `o` yazar.

6.3 Escape sembolünün seçimi

Yeni bir problem: Metinde bir e-posta adresi varsa `@` metinde geçer. Bu yüzden önce bir ön işlemle metinde olmayan ama ASCII tablosunda bulunan bir karakter bulunur ve escape olarak o kullanılır. Karşı tarafın hangi karakterin escape olduğunu bilmesi için başa bir başlık eklenir. Örneğin başlığın ilk baytına kullanılan escape karakteri yazılır. Ya da başlığa iki bayt konur: birinci bayt 0 ise escape kodu yoktur, 1 ise ikinci bayt escape kodudur.

6.4 En az kaç tekrar?

Bazen sıkıştırılmış veri orijinalinden uzun olabilir. Hoca run ve karakteri sayarak anlattı: Tek a için 1a yazmak 1 baytı 2 bayta çıkarır. aa için 2a yazmak 2 baytı yine 2 bayt yapar, sıkıştırma yoktur. aaa için 3a yazmak 3 baytı 2 bayta indirir, sıkıştırma vardır. Bu yüzden RLE'nin bir varyasyonunda karakter en az 3 kez tekrar ediyorsa run'lı (3a) yazılır; 3'ten az tekrarlar olduğu gibi yazılır (aa ise aa). Başka versiyonlar en az 4 tekrar ister.

> **Ders sonrası not:** Bu sayımda escape sembolü hesaba katılmamıştır. Escape kullanılan biçimde (@3a) run'lı yazım 3 bayttır; o zaman 3 tekrarda kazanç olmaz, kazanç 4 tekrardan itibaren başlar. Escape'li ve escape'siz versiyonlarda eşiğin farklı seçilmesinin nedeni budur.

6.5 Run'ın 3 tekrardan sonra başladığı versiyon

Bir versiyonda run değeri, 3 tekrardan sonra gelen karakter sayısını gösterir. Dördüncü a gelince run'lı yazılır ve run değeri 1 olur. Açıcı, run'lı yazıldığına göre önce 3 tekrar olduğunu bilir ve 1'in üzerine 3 ekleyip 4 a yazar. Bunun amacı bir baytı daha iyi kullanmaktır: Bir baytta en fazla 255 yazılır; bu versiyonda 255 değeri $255+3 = 258$ tane a anlamına gelir. Hoca bunları "ufak ufak oyunlar" olarak niteledi.

7. RLE'nin Temel Problemleri ve Versiyonları

Slayttaki temel problemler (ekrandan, 01:00:00):

- Doğal dilde yazılmış metinlerde arka arkaya tekrar eden karakter nadirdir (boşluklar hariç). Bu yüzden metinde RLE kullanmak pek mantıklı değildir.
- Tekrar sayısına 1 bayt ayrılmıştır ve 255 ile sınırlıdır. Tekrar sayısı 3 tekrar karakterden sonra kullanılacaksa 258 adet aynı karakter 3 bayt ile kodlanır.
- Kullanılan özel escape kodu metin içinde geçebilir; farklı bir karakter seçilmelidir.
- Exe türü dosyalarda bütün ASCII karakterler dosyada yer alabilir. Bu durumda farklı bir yol izlenmelidir.

7.1 260 tane a

Hoca 260 tane a'nın nasıl yazılacağını sordu. Standart versiyonda bir run en fazla 255 olabildiği için önce 255 a, sonra kalan 5 a için ikinci bir run yazılır: $255 + 5 = 260$. Run'ın 3 tekrardan sonra başladığı versiyonda ise 255 run değeri 258 a demektir; geriye 2 a kalır ve bunlar run kullanılmadan aa olarak yazılır, çünkü 3'ten az tekrar run'la yazılmaz. Hangi versiyon kullanıldıysa açıcı da ona göre yazılacağı için problem olmaz.

7.2 Exe dosyaları

Exe dosyalarında bütün ASCII karakterler kullanılmış olabilir, escape olarak seçilecek boş karakter kalmaz. Run değerinin başladığını göstermek için başka bir yol gerekir. Hoca bunun için ileride ayrıca göreceğimiz MNP5 algoritmasındaki RLE versiyonunu gösterdi.

8. MNP5 ve RLE ile Karşılaştırılması

8.1 MNP5

Hoca bir yöntemin illa tek başına kullanılması gerekmediğini yeniden hatırlattı: JPEG'te de DCT, RLE ve bir kodlama adımı arka arkaya gelir.

MNP5, Microcom Network Protocol'de uygulanan bir yöntemdir; hoca fakslarda da kullanıldığını söyledi. Slayttaki kural (ekrandan, 01:02:10): Input stream'de 3 karakter arka arkaya tekrar ediyorsa çıktıya 3 kopya yazılır; dördüncü bayt run değeridir. Escape sembolüne gerek yoktur, çünkü üç aynı karakter görmek arkasından run geleceğinin işaretidir.

Girdi	Çıktı	Sonuç
`aaa`	`aaa0`	3 bayt yerine 4 bayt (genişleme)
`aaaa`	`aaa1`	4 bayt yerine 4 bayt (sıkıştırma yok)
`aaaaa`	`aaa2`	5 bayt yerine 4 bayt (sıkıştırma var)

Üç a için run değeri 0'dır, çünkü 3 a zaten yazılmıştır. Dört a için 3'ün üzerine 1 eklenir. Tekrar sayısı 5'in üzerine çıktıkça sıkıştırmadan söz edilebilir; aksi durumda sıkıştırma yoktur.

8.2 Performans karşılaştırması

Hoca ara vermeden önce bir örnek daha verdi (ekrandan, 01:06:50). N: dokümandaki karakter sayısı, M: tekrar eden grup sayısı (grup, arka arkaya tekrar eden karakterler), L: gruplardaki ortalama eleman sayısı. Sıkıştırma oranı "çıktı boyu / girdi boyu"dur. Her iki yöntemde girdi N'dir.

- Standart RLE (escape + run + karakter): M grubun toplam ML karakteri çıkarılır, her grup yerine 3 bayt eklenir.
- MNP5: Aynı karakterler çıkarılır, her grup yerine 3 tekrar + 1 run = 4 bayt eklenir.

$$\text{RLE} = \frac{N - ML + 3M}{N}, \qquad \text{MNP5} = \frac{N - ML + 4M}{N}$$

$N = 1000$, $M = 10$, $L = 5$ için:

$$\text{RLE} = \frac{1000 - 50 + 30}{1000} = 0{,}98 \quad (\%2 \text{ sıkıştı})$$

$$\text{MNP5} = \frac{1000 - 50 + 40}{1000} = 0{,}99 \quad (\%1 \text{ sıkıştı})$$

RLE 100 birimi 98'e, MNP5 99'a indirmiştir. Hoca buradan "RLE MNP5'ten daha iyidir" denemeyeceğini vurguladı: Veriler değişince MNP5 daha iyi olabilir.

9. Görüntüde RLE: Piksel, Renk Derinliği ve Nicemleme

Aradan sonra hoca görüntü sıkıştırmaya geçti. Mantık metindekiyle aynıdır, ama veri RLE'de kullanılacak forma getirilirken bazı şeylere dikkat edilir.

Metindeki sembol ya da karakter, görüntüde piksele karşılık gelir; her renk koduna piksel denir.

- Siyah-beyaz (black-white) resimde bir piksel 1 bitle kodlanır.
- Grayscale resimde bit sayısı derecesine göre değişir: 2 bitlik grayscale 4 farklı ton demektir; 3 bitlik, 7 bitlik grayscale de olabilir.
- Color table resimde tabloya 256 farklı renk atanmıştır (0-255); her piksel 8 bittir.
- True color resimde red, green, blue diye 3 palet vardır; her biri 8 bit, toplam $3 \cdot 8 = 24$ bittir. Bu 2^{24} farklı renk demektir.

Göz 2^{24} rengin hepsini ayırt edemediği için RLE'den önce mutlaka bir nicemleme (quantization) işlemi yapılır. Nicemleme renk sayısını azaltmaktır. Örneğin (R 50, G 50, B 49) ve (R 50, G 50, B 48) iki farklı renktir, ama göz bu farkı algılayamaz; ikisi tek renge indirilir.

10. Bitmap'in Taranması ve Siyah-Beyaz Resimde RLE

10.1 Scanline'lar ve başlangıç noktası

Bir bitmap'te pikseller scanline'lara (satırlara) yerleşir; örneğin 512 × 512 çözünürlüklü bir resim. Satırlar ve dikey sütunlar bir ızgara oluşturur; kesişim noktalarında piksel değerleri vardır. İlk piksel resmin sol üst köşesi, son piksel sağ alt köşesidir.

RLE ile resim sıkıştırırken işleme nereden başlanacağı önemlidir. Hoca boyalı bir şekil örneği verdi: Satır satır ya da sütun sütun düzenli okununca arka arkaya hep kırmızılar gelir. Rastgele bir noktadan okumaya başlanırsa önce kırmızının küçük bir kısmı, sonra beyaz, sonra yine kırmızı gelir; renklerin sürekliliği kesilir ve sıkıştırma kötüleşir.

10.2 Tarama biçimleri

Slayta göre kompresör bitmap'i satır satır, sütun sütun veya zigzag olarak tarar (ekrandan, 01:40:00). Satır satır okumada önce birinci satır, sonra ikinci satır okunur ve hepsi tek bir girdi olarak RLE'ye verilir. Sütun sütun okumada aynı işlem sütunlar için yapılır.

Bir de rotate'li (dönerek) okuma vardır: Bir satır soldan sağa okununca alttaki satır sağdan sola okunur. Hoca örnek olarak iki satırın sağ uçlarının boyalı olduğu bir şekil çizdi. Düz okumada birinci satırın sonundaki kırmızılar ve ikinci satırın sonundaki kırmızılar ayrı run'lar olur. Rotate'li okumada birinci satırın sonundaki kırmızılar ikinci satırın başına (sağ ucuna) bağlanır ve tek uzun bir run oluşur; 20 kırmızıyı ve 20 kırmızıyı ayrı ayrı kodlamak yerine 40 kırmızı tek seferde kodlanır. Resimdeki nesneler birkaç satır boyunca devam ettiği için alt satır da aynı renkte olabilir; bu yüzden rotate'li okuma daha etkili sıkıştırma verebilir.

10.3 Siyah-beyaz resimde kurallar

Slayttaki kurallar (ekrandan, 01:40:00):

- Kompresör ilk okunacak pikselin beyaz olduğunu kabul eder.
- İlk piksel beyaz değilse çıktıya önce 0 adet beyaz yazılır.
- Resim 17 beyaz, onu izleyen 1 siyah ve 55 beyaz pikselden oluşmuşsa çıktıya `17 1 55` yazılır.
- Çıktının başına bitmap'in çözünürlük değeri yazılır.

Siyah-beyaz resimde renk kodu yazmaya gerek yoktur, çünkü piksel ya siyahtır ya beyazdır. İlk run'ın beyaz olduğu kabul edilince yalnız run değerleri yazılır ve renkler sırayla değişir. Örneğin `65 72`, 65 beyaz ve 72 siyah demektir. Resim siyahla başlıyorsa `0 65 72` yazılır: 0 beyaz, 65 siyah, 72 beyaz.

Çözünürlüğün yazılması önemlidir: Açarken bir satıra kaç piksel konacağı, kaç scanline ve kaç sütun olduğu bilinmelidir. Bu yüzden başa satır ve sütun değerleri (x, y) yazılır.

10.4 Şeklin karmaşıklığı

Sıkıştırılmış akışın büyüklüğü şeklin karmaşıklığına bağlıdır; şekilde ne kadar fazla detay varsa sıkıştırma oranı o kadar kötüdür. Yarısı kırmızı, yarısı mavi bir resim çok iyi sıkışır. Masmavi bir gökyüzü ve yeşillikler de iyi sonuç verir. Renklerin birbirine çok karıştığı, sürekliliğin az olduğu bir resimde run değerleri küçük olur ve RLE başarısızdır.

10.5 Uniform alanda sıkıştırma oranı

Slayta göre uniform bir alanda RLE için sıkıştırma oranı, şeklin çevre uzunluğunun yarısının alan içindeki toplam piksel sayısına oranıdır. Şekil kare, daire ya da üçgen gibi düzgün olmayabilir; çevresi nasıl hesaplanır? Resim scanline'lardan oluştuğu için şekli baştan sona kaç scanline'ın taradığına bakılır. Her scanline'ın şekle bir giriş ve bir çıkış noktası vardır; bu yüzden $2\times$ scanline sayısı uniform şeklin çevre uzunluğunu (çevresindeki piksel sayısını) verir (ekrandan, 01:50:00).

$$\text{oran} = \frac{\text{çevre uzunluğu}/2}{\text{alandaki toplam piksel sayısı}}$$

Hoca anlatırken "yarısı" kelimesinden şüphe edip "iki katı" olabileceğini düşündü, slaytta kelimenin üstünü çizdi; sonra kaynak kitaba bakıp "yarısıymış" dedi. Hoca pratikte bir resimdeki nesnenin sıkıştırmaya katkısını böyle ölçmeyeceğimizi, resmin tamamı için çıktı/girdi oranına bakacağımızı ekledi.

11. Renkli Resim Örnekleri ve Run Uzunluğunun Bit Sayısı

11.1 Beş renkli resim

Hoca internetten aldığı bir örneği gösterdi (ekrandan, 01:53:00). Resim 15 × 11 piksel boyutundadır; palette 16 renk vardır. Şekilde 5 renk kullanılmıştır (1, 5, 7, 11, 15 numaralı renkler). 16 renk 0-15 arası 4 bitle kodlanabildiği için sıkıştırmasız boyut:

$$15 \cdot 11 \cdot 4 = 660 \text{ bit}$$

RLE ile her scanline satır satır okunur. Birinci satırda 15 tane 5 vardır: `15 5`. İkinci satırda 7 tane 5, 4 tane 11, 4 tane 5: `7 5 4 11 4 5`. Her çiftin ilki run, ikincisi renk kodudur. Bütün satırlar okununca 134 sayı çıkar ve slayt sonucu şöyle verir:

$$134 \cdot 4 = 536 \text{ bit}$$

11.2 Neden 4 ile çarpıldı?

Hoca burada bir soru sordu: Renk kodları 4 bittir, ama run'lar için 1 bayt demiştik. 134 sayının yarısı run ise neden hepsi 4 ile çarpıldı, run'lar 8 ile çarpılmalı değil miydi?

Cevap: Algoritmayı ve açıcısını yazan biz olduğumuz için her türlü kararı verebiliriz. Bu resimdeki run değerlerine bakılınca (15, 3, 2, 1, 7, 11 …) en büyüğünün 15 olduğu görülür. 15, 4 bitle yazılabilir; 4 bit yetiyorsa 8 bit harcamanın anlamı yoktur. Run değerleri 64'ten küçükse 6 bit, 128'den küçükse 7 bit yeter. Run'a illa 1 bayt ayrılacak diye bir kural yoktur.

> **Ders sonrası not:** 8 bit 0-255 aralığını tutar; 128-255 arası run'lar için 8 bit gerekir. 256 ve üzeri bir run tek bir 8 bitlik alana sığmaz; ya daha fazla bit ayrılır ya da run bölünür. Altyazıdaki "256'nın üzeriyse 8 bayt" ifadesi büyük olasılıkla "8 bit" demek istemiştir.

Açıcının run'lar için kaç bit ayrıldığını bilmesi gerekir; bu bilgi başlığa yazılır. Açıcı başlıktan run'ların 4 bit olduğunu okur, ilk 4 biti run, arkasındaki 4 biti renk kodu olarak alır. Yani run'ların ve renklerin kaç bitte tutulduğu başlığa yazılmalıdır.

Run'a da renge de her zaman 1 bayt verilirse redundancy (artıklık) oluşur: Veride hâlâ sıkıştırılabilecek bir şey vardır ama gereğinden fazla bit kullanıldığı için sıkıştırılmamıştır.

11.3 Renk sayısını azaltmak (kayıplı)

Aynı resmin 15 numaralı rengi kaldırılmış, daha az renkle ifade edilmiş hâli (ekrandan, 01:58:50) 3 renk içerir (5, 11, 1). Renk sayısı 5'ten 3'e inince run'lar uzar:

```
15 5
7 5 4 11 4 5
4 5 2 11 1 1 6 11 2 5
3 5 3 11 2 1 6 11 1 5
2 5 4 11 2 1 6 11 1 5
1 5 5 11 3 1 5 11 1 5
1 5 6 11 2 1 5 11 1 5
1 5 6 11 2 1 4 11 2 5
2 5 5 11 2 1 3 11 3 5
3 5 7 11 5 5
15 5
```

Toplam 86 sayı vardır (python ile sayıldı): $86 \cdot 4 = 344$ bit. 536 bitten 344 bite düşülmüştür. Siyah-beyaz resimlerde daha iyi sonuç alınmasının nedeni de budur: renk sayısı azaldıkça run'lar uzar.

Hoca bunun kayıplı bir dönüşüm olduğunu belirtti. Örneğin bir gökyüzünde mavinin çok değişik tonları varsa birbirine yakın maviler tek maviye indirilebilir. Bu örnekte açık yeşil ile koyu yeşil, 15 ile 1 numaralı renkler aynı renge çevrilmiştir. Önemli olan şeklin kaybolmamasıdır; run'lar daha düzgün ve anlamlı olur, sıkıştırma çok daha iyi çıkar.

11.4 Çok karışık resim

Son örnekte (ekrandan, 02:00:00) neredeyse her pikselde renk değişir: turuncu, sarı, turuncu, sarı … Her run 1'dir; 4 bitte tutulacak bir pikseli 8 bitle (4 bit run + 4 bit renk) saklamak gerekir. Sonuç:

$$311 \cdot 4 = 1244 \text{ bit}$$

660 bitlik resim 1244 bite çıkmıştır; sıkıştırma değil genişleme vardır. Hoca bu noktada ilk haftalarda söylediği şeye döndü: Onlarca sıkıştırma yöntemi vardır. Verinin içeriği, kaybın kabul edilip edilmediği, hızlı çalışma mı iyi sıkıştırma mı istendiği, sıkıştırma ve açma sürelerinden hangisinin kısa olması gerektiği gibi parametrelere göre en uygun yöntem seçilir. "Bir yöntem çok iyi, her şeyde onu kullanayım" denmemelidir.

12. Grayscale ve True Color Resimlerde RLE Versiyonları

12.1 Problem

8 bitlik grayscale bir resim olsun (4 ya da 7 bitlik de olabilir). Yine run değeri ve arkasından pikselin renk kodu yazılır. Girdi (ekrandan, 02:07:10):

```
12 12 12 12 12 12 12 12 12 35 76 112 67 87 87 87 5 5 5 5 5 5 1 ...
```

9 tane 12, birer tane 35, 76, 112, 67, 3 tane 87, 6 tane 5 ve 1 vardır. Run değeri 1 olanlar için boşuna run yazmamak üzere yalnız birden fazla tekrar edenlerin yanına run yazılırsa açıcı bir sayının run mı renk kodu mu olduğunu bilemez. Örneğin 9 bir renk kodu mu, run mı?

12.2 Yöntem I: işaret biti ya da 255 işareti

Slayttaki birinci yöntem (ekrandan, 02:07:10): Görüntü 128 grayscale ile sınırlıysa (0-127), her baytın soldaki biti "grayscale değeri mi, tekrar sayısı mı" diye kullanılır. 128 renk 7 bitle yazılır ($2^7 = 128$). Sayı 7 bitte tutulur, 8. bit (en soldaki) 0 ise okunan 7 bit renk kodu, 1 ise run değeridir. 6, 4, 3 bitlik grayscale resimlerde de aynı yol izlenebilir.

Hoca slaytta "128 grayscale" ifadesini önce yanlış yazdığını düşünüp sayıları kontrol etti ve bunun 7 bitlik grayscale resim olduğunu söyledi.

Resimde 0-255 arası bütün renkler kullanılıyorsa boş bit kalmaz. Bu durumda değer bir azaltılarak kullanılır: 255 renk değeri 254 olarak alınır. Bu küçük bir kayıp gibidir, ama 254 ve 255 birbirine çok yakın renklerdir. Artık 255 sayısı renk olarak kullanılmadığı için "arkasından run gelir" işareti olur:

```
Compressed: 255 9 12 35 76 112 67 255 3 87 255 6 5 1 ...
```

255'ten sonra gelen 9 run, 12 renktir. 35, 76, 112, 67 normal renk değerleridir. 255 3 87: 3 run, 87 renk. 255 6 5: 6 run, 5 renk. Sonra 1 normal renk değeridir.

12.3 Yöntem II: bayrak baytı

Slayttaki ikinci yöntem (ekrandan, 02:09:10): Her baytın ne olduğunu gösteren bir bit kullanılır. Bu ekstra bitler 8'lik gruplar hâlinde tutulur; çıktıya tekrar ve renk değerleri gönderilmeden önce bu bilgiye ait flag değerlerini tutan 1 baytlık değer gönderilir. Her 8 bayt için fazladan 1 bayt gönderilir.

```
Compressed: 10000010 9 12 35 76 112 67 3 87 100... 6 5 1 ...
```

İlk bayrak baytı sonraki 8 değeri tanımlar: 9 için 1 (run), 12, 35, 76, 112, 67 için 0 (renk), 3 için 1 (run), 87 için 0 (renk). Sonraki 8'lik grubun bayrağı `100...` ile başlar: 6 run, 5 ve 1 renk. Slaytta bu grubun gerisi yarım bırakılmıştır.

12.4 True color resim

True color resimlerde her piksel 3 bayt (R, G, B) ile gösterilir. Slayttaki 4 piksel (ekrandan, 02:10:00):

```
(171 85 34) (172 85 35) (172 85 30) (173 85 33)
```

Red değerleri birbirine çok yakındır; bunlar kırmızının dört tonudur. Hoca burada ortalama alınarak kayıplı sıkıştırma da yapılabileceğini söyledi ama ayrıntısına girmedi. RLE her palet için ayrı ayrı uygulanır:

```
R: (171 172 172 173)
G: (85 85 85 85)
B: (34 35 30 33)
```

Önce red değerleri RLE'ye gönderilir, sonra green, sonra blue; sonuçlar arka arkaya eklenir. Açarken her palet kendi içinde ayrı ayrı açılır. Kodlayan da açan da biz olduğumuz için bu tercih bize kalmıştır.

13. Tarama Yönünün Seçimi ve Zigzag Okuma

13.1 Satır mı, sütun mu?

Bir bitmap satır satır mı, sütun sütun mu, yoksa zigzag mı okunmalı? Hoca resmin içeriğinin bu konuda çok şey söylediğini belirtti. Ekrana I harfi gibi dikey çizgilerden oluşan bir şekil düşünülürse satır satır okumada küçük küçük run'lar oluşur; sütun sütun (ve rotate'li) okumada çok daha iyi sıkıştırma elde edilir.

> **Hocanın vurgusu:** Resim hep dikey çubuklardan oluşuyorsa onu satır satır okumak "büyük bir aptallıktır". Renkler hep yatay gidiyorsa sütun sütun okumak da öyledir. Satır mı sütun mu kararı resmin içeriğine göre verilir.

13.2 Zigzag okuma

Zigzag okuma örneğin JPEG'te kullanılır. Hoca güneş, bulut, kırmızı pencereli ve damlı bir ev, bir ağaç bulunan bir resim çizdi. Çok fazla nesne ve rengin olduğu böyle bir resimde satır satır da sütun sütun da okusak başarı elde edemeyiz.

Bu yüzden resim parçalara bölünür: 8 × 8, 16 × 16, 32 × 32 ya da 64 × 64 bloklar. JPEG genelde 8 × 8 bloklar kullanır. Güneşten 8 × 8 bir blok alınırsa bloğun tamamı sarıdır. Bir pikselin 8 komşusu vardır (kuzey, güney, doğu, batı, kuzeydoğu, kuzeybatı, güneydoğu, güneybatı); pikselin ve komşularının renk kodları genelde aynı ya da birbirine yakındır. Ani geçişler yalnız keskin kenarlarda olur. Renklerdeki bu yumuşaklığı yakalamak için resmin tamamı yerine küçük bloklar alınır ve her blok zigzag okunur: 1 numaralı pikselden 2'ye, oradan çaprazla 3'e, 4'e … diye bloğun sonuna kadar gidilir. Zigzag okumanın sonunda yine düz bir dizi ortaya çıkar ve buna RLE uygulanır.

Zigzag okuma çok karışık desenlerin, motiflerin ve çok fazla rengin olduğu resimlerde tercih edilir. Resmin tamamına zigzag uygulanmaz; o zaman yine başarısız olunur. Bloklarla çalışınca satır ve sütun seçimine gerek kalmaz.

13.3 Siyah-beyaz örnek: row-row ve column-column

Hoca 8 × 8 bir siyah-beyaz resim üzerinde iki okumayı karşılaştırdı (ekrandan, 02:20:00). Resimde üstte 4 satır boyunca beyaz-siyah-beyaz-siyah-siyah-beyaz-siyah-beyaz dikey çizgiler, altında 7 beyaz + 1 siyah satır, tamamen siyah bir satır, yine 7 beyaz + 1 siyah ve tamamen siyah bir satır vardır. Her satırın sonuna bir satır sonu işareti (`<eoln>`) konur.

Row-row (satır satır):

```
1 1 1 2 1 1 1 <eoln>   -> 8 x 4 = 32
7 1 <eoln>             -> 3 x 2 = 6
0 8 <eoln>             -> 3 x 2 = 6
                          44 bayt
```

Resim beyazla başladığı için ilk satır doğrudan 1 beyazla başlar. Tamamen siyah satırlar beyazla başlamadığı için `0 8` yazılır: 0 beyaz, 8 siyah.

Column-column (sütun sütun):

```
5 1 1 1 <eoln>          -> 5 x 3 = 15
0 4 1 1 1 1 <eoln>      -> 7 x 4 = 28
4 4 <eoln>              -> 3 x 1 = 3
                           44 bayt (slayttaki değer)
```

Siyahla başlayan sütunlarda yine önce 0 beyaz yazılır. Hoca sonuçların aynı çıktığını, çünkü çok küçük bir parça alındığı için satır ve sütun okuma arasında fark olmadığını söyledi.

> **Ders sonrası not:** Slaytta column-column toplamı da 44 bayt yazıyor. Slayttaki satırlarla toplam $15 + 28 + 3 = 46$ bayttır (python ile kontrol edildi). Bu örnekte satır satır okuma 2 bayt daha iyidir.

Hoca bu resim rotate'li okunsaydı bazı şeylerin değişeceğini söyledi: Satır sonundaki siyah, alt satıra dönülünce oradaki siyahlarla birleşir ve run'lar uzar. Bir kısmında row, bir kısmında column daha iyi sonuç verecekti, ama rotate'li okumanın her zaman daha iyi sonuç verdiğini biliyoruz.

Bu resim zigzag için 4 × 4 bloklara da bölünebilirdi.

13.4 Dolgu (padding)

Bloklar 4 × 4 ya da 8 × 8 alınıyorsa ve resmin boyutu blok boyunun katı değilse ne olur? Hocanın örneğinde 4'er 4'er ayrılınca sonda 3 sütun kalıyordu; o zaman bir dolgu (padding) sütunu eklenir. Dolgu illa sıfırla doldurmak değildir: En sondaki piksel değerleri neyse aynı değerler dolgu olarak konur. Böylece sıkıştırma sırasında yine tekrar eden renkler oluşur.

14. Kayıplı RLE ve RLE'nin Özeti

14.1 Kayıplı RLE

RLE'de kayıplı sıkıştırma da yapılabilir. Slayttaki maddeler (ekrandan, 02:24:30):

- Görüntü sıkıştırmada daha iyi oranlar elde etmek istiyorsak bazı ihmallerde bulunmamız gerekir.
- Kayıplar kullanıcıya göre bazı uygulama alanlarında kabul edilebilir.
- Kısa olan run değerleri ihmal edilebilir.
- Kullanıcıya ihmal edilebilecek en büyük run uzunluğu sorulur. Kullanıcı üç demişse 1, 2 ve 3 uzunluğundaki piksel değerlerinin yerine en yakın komşu pikselin değeri kabul edilir.

Hocanın tahtadaki örneğinde renk değerleri hep 64 olarak gider, arada tek bir 63 kalmıştır. RLE bunu "bir tane 64, bir tane 63, iki tane 64" gibi ayrı run'larla yazar. Göz 63 ile 64'ü rahat ayırt edemez. 63'ün run değeri 1'dir; sağında ve solunda 64 vardır. Kısa run ihmal edilir: sağında ve solunda hangi renk daha fazlaysa o renge çevrilir, yani 63 yerine 64 yazılır. Açınca orijinal elde edilmez, ama resmin anlaşılırlığı bozulmaz, zaten göz farkı ayırt edemez.

14.2 Özet

Hoca RLE'yi şöyle özetledi:

- RLE aslında kayıpsız bir sıkıştırma yöntemidir. En yüksek başarı run değerlerinin en uzun olduğu resimlerde ya da dosyalarda alınır. Kayıplı olması gerekmez; tercihe bağlıdır.
- Resim dosyası da olsa metin de olsa veri bit formuna döküldükten sonra satır satır, sütun sütun, rotate'li, küçük bloklara bölünerek ya da zigzag okunabilir.
- Kodlaması çok kolay, basit bir algoritmadır; çok fazla varyasyonu vardır.
- Veri çok karışıksa, aralarda sürekli tek karakter geçiyorsa ve bunlar için de run kullanılıyorsa, farklı bir versiyon kullanılmıyorsa ya da resimde birbirine yakın renkler için ihmal yapılıp kayıplıya dönülmüyorsa negatif sıkıştırma olur.

Örnek Sorular

Soru 1 (ekrandan, 00:10:00 ve 00:20:00): `Bu__dersin__adı___veri_sıkıştırma___ve_pazartesi_günü__ya` metnine (_ boşluk) sparse string sıkıştırma uygulayın. L1, L2, L3 seviyelerini, sıkıştırılmış seviyeleri ve başlığı yazın.

> **Çözüm (derste verildi):** Karakter 0, boşluk 1: `L1 = 0011|0000|0011|0001|1100|0010|0000|0000|0111` Sıfırdan farklı blok 1: `L2 = 1011|1100|1100|1100`, `L3 = 1111`. Sıfır bloklar silinince `L1 = 0011|0011|0001|1100|0010|0111|0010|1000|0110` (36 bit), L2 16 bit, L3 4 bit. Başlık: `3 36 16 4`; ardından L1, L2, L3 ve boşluksuz metin `budersinadıverisıkış-tırmavepazartesigünüyapılıyor` gönderilir.

Soru 2 (ekrandan, 00:30:00): Baudot kodlamasıyla `AB46` dizisini kodlayın. Kodlar LS ve FS olmadan gönderilirse açıcı nasıl bir problemle karşılaşır?

> **Çözüm (derste verildi):** A `10000`, B `00110`, 4 `10100`, 6 `10010`. LS ve FS yoksa `10000` tabloda hem A hem 1'e karşılık geldiği için açıcı hangisi olduğunu bilemez. Doğru kodlama: `00001 10000 00110 00010 10100 10010` (LS A B FS 4 6).

Soru 3 (ekrandan, 00:32:30): Sırasıyla `a, aardvark, aback, abaft, abandon, abandoning, abasement, abandonment, abash, abated, abate, abbot, abbey, abbreviating, abbreviate, abbreviation` kelimelerinden oluşan listeyi önek sıkıştırma ile kodlayın.

> **Çözüm (derste verildi, düzeltmeyle):** Her kelime üstündeki kelimeyle soldan ortak karakter sayısı + kalan ek olarak yazılır: `a`, `1ardvark`, `1back`, `3ft`, `3ndon`, `7ing`, `3sement`, `3ndonment`, `3sh`, `3ted`, `5`, `2bot`, `3ey`, `3reviating`, `9e`, `9ion` (python ile kontrol edildi). Slaytta `abandoning` için `3ndoning`, `abate` için `3te` yazıyor; bunlar bir üstteki kelimeyle uyuşmaz (Bölüm 4'teki düzeltme). `abbreviate` 10 baytı 2 baytla yazar.

Soru 4 (ekrandan, 00:43:40): `aaaaabbbbbbbbbbbbccccddddddddddeeeeeeeeee` ve `aabccdeefg-hijjjklmnopqrrstttuvvwwxyyyz` dizilerine RLE uygulayın. Hangisi sıkışır, hangisi sıkışmaz? Neden?

> **Çözüm (derste verildi):** Birincisi `5a12b4c9d10e`: 40 bayt 10 bayta iner. İkincisi `2a1b2c1d2e1f1g1h1i3j1k1l1m1n1o1p1q2r1s3t1u2v2w1x3y1z`: 38 bayt 52 bayta çıkar (negatif sıkıştırma). İkili tekrarlarda kazanç yoktur, tek karakterler 1 bayttan 2 bayta çıkar. RLE'nin başarısı tekrarların uzunluğuna bağlıdır.

Soru 5 (ekrandan, 00:50:00; derste sözlü 00:51:08): `2. all is too well` metni `2. a2l is t2o we2l` olarak sıkıştırılmıştır. Burada sıkıştırma var mı? Açıcı ikileri doğru anlar mı? Problem nasıl çözülür?

> **Çözüm (derste verildi):** Sıkıştırma yoktur; her iki karakterlik tekrar yine 2 baytla yazılmıştır. Açıcı ortadaki `2l` ve `2o`'yu anlar, ama baştaki 2'yi run sanıp iki nokta yazabilir. Çözüm run'lardan önce metinde geçmeyen bir escape sembolü (derste `@`) koymaktır: `2. a@2l is t@2o we@2l`. Hangi karakterin escape olduğu başlığa yazılır.

Soru 6 (derste sözlü, 00:58:33 - 00:59:39; ekrandan, 01:00:00): Run değerine 1 bayt ayrılmışsa 260 tane `a` nasıl yazılır? Run'ın 3 tekrardan sonra başladığı versiyonda ne değişir?

> **Çözüm (derste verildi):** Bir run en fazla 255'tir: 255 `a` ve 5 `a` olarak iki run yazılır. Run'ın 3 tekrardan sonra başladığı versiyonda 255 değeri 258 `a` demektir; kalan 2 `a`, 3'ten az olduğu için `aa` olarak yazılır.

Soru 7 (ekrandan, 01:06:50): $N = 1000$ karakterlik bir girdide $M = 10$ tekrar eden grup ve her grupta ortalama $L = 5$ eleman var. Standart RLE (escape + run + karakter) ile MNP5'ten hangisi daha başarılıdır?

> **Çözüm (derste verildi):** RLE: $(1000 - 50 + 30)/1000 = 0{,}98$, %2 sıkıştırma. MNP5: $(1000 - 50 + 40)/1000 = 0{,}99$, %1 sıkıştırma. Bu veride RLE daha iyidir, ama veri değişince MNP5 daha iyi olabilir; genel bir üstünlük söylenemez.

Soru 8 (derste sözlü, 01:52:18 - 01:54:34; ekrandan, 01:53:00): 16 renkli, 15 × 11 piksellik resim RLE ile 134 sayıya kodlanmış ve boyut $134 \cdot 4 = 536$ bit olarak hesaplanmıştır. Sayıların yarısı run, run'lar için 1 bayt denmişti. Neden hepsi 4 ile çarpıldı?

Çözüm (derste verildi): Bu resimdeki en büyük run 15'tir ve 4 bitle yazılabilir. Algoritmayı ve açıcıyı yazan biz olduğumuz için run'a 8 bit yerine 4 bit ayırabiliriz; run ve renk bit uzunlukları başlığa yazılır. Gereğinden fazla bit kullanmak artıklık (redundancy) oluşturur.

Soru 9 (ekrandan, 02:07:10 ve 02:09:10): `12 12 12 12 12 12 12 12 12 35 76 112 67 87 87 87 5 5 5 5 5 5 1` grayscale dizisinde 1 olan run'lar yazılmayacaktır. Run ile renk kodunu ayırt etmek için (a) 255 işaretli yöntemle, (b) bayrak baytı yöntemiyle sıkıştırın.

Çözüm (derste verildi): (a) Renkler 0-254'e indirilir, 255 "arkasından run gelir" anlamındadır: `255 9 12 35 76 112 67 255 3 87 255 6 5 1`. (b) Her 8 değer için önce bir bayrak baytı (1 = run, 0 = renk): `10000010 9 12 35 76 112 67 3 87`, ardından `100...` bayrağı ve `6 5 1`.

Soru 10 (ekrandan, 02:20:00): 8 × 8 siyah-beyaz resmi row-row ve column-column okuyarak RLE ile kodlayın, bayt sayılarını karşılaştırın.

Çözüm (derste verildi, düzeltmeyle): Row-row: `1 1 1 2 1 1 1 <eoln>` × 4 = 32, `7 1 <eoln>` × 2 = 6, `0 8 <eoln>` × 2 = 6; toplam 44 bayt. Column-column: `5 1 1 1 <eoln>` × 3 = 15, `0 4 1 1 1 1 <eoln>` × 4 = 28, `4 4 <eoln>` × 1 = 3. Slayt ve hoca toplamı 44 bayt diye verdi; doğru toplam $15 + 28 + 3 = 46$ bayttır. Rotate'li okuma run'ları birleştirerek sonucu iyileştirebilir.

Soru 11 (derste sözlü, 02:27:50 - 02:31:16; uygulama ödevinin akademik metni): Bir siyah-beyaz, bir 3 bitlik grayscale ve bir 256 renkli (color table) resim alın. Üçüne de istediğiniz bir RLE versiyonunu (örneğin MNP5'teki gibi 3 tekrardan sonra run yazan ya da run'ları bir bayrakla gösteren) uygulayın. Resimleri row-row ve row-rotate (ya da column-column ve column-rotate) okuyun, ayrıca zigzag okuyun ve sıkıştırma performanslarını bir tabloda karşılaştırın.

Çözüm (derste verilmedi): Bu bir uygulama ödevidir; sonuç seçilen resimlere ve RLE versiyonuna bağlıdır, tek bir sayısal cevabı yoktur. Hoca row ve rotate okuma kodlarının öğrencilerce yazılmasını, zigzag okuma için hazır kod kullanılabileceğini söyledi.

Dersin Toparlaması

Bu derste geçen haftadan kalan sparse string sıkıştırma tamamlandı: karakterlere 0, boşluklara 1 verilerek seviyeler oluşturuldu, sıfır bloklar silindi, başlığa seviye sayısı ve bit uzunlukları yazıldı ve açma işleminin L3'ten başlayıp tersten nasıl yapıldığı gösterildi. Ardından iki ad hoc yöntem daha görüldü: harf ve rakamları LS/FS kodlarıyla ayıran 5 bitlik Baudot kodlaması ve alfabetik sözlükleri ortak öneklerle sıkıştıran önek sıkıştırma.

Dersin büyük kısmı Run Length Encoding'e ayrıldı. RLE'nin `nd` çifti fikri, metinde neden başarısız, büyük tek renkli bölgeleri olan resimlerde neden başarılı olduğu örneklerle gösterildi. Run ile veriyi ayırt etmek için escape kodu, 255 sınırı, 3 tekrardan sonra başlayan versiyon ve escape gerektirmeyen MNP5 incelendi; ikisinin performansı formülle karşılaştırıldı. Görüntüde piksel ve renk derinliği, nicemleme, satır, sütun, rotate'li ve zigzag tarama, siyah-beyaz resimde ilk pikselin beyaz kabul edilmesi, run'lar için bit sayısının veriye göre seçilmesi, grayscale ve true color resimler için RLE versiyonları ve kısa run'ları ihmal eden kayıplı RLE anlatıldı. Hoca sonraki derste bilgi teorisiyle (information theory) başlanacağını ve istatistiksel yöntemlere geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.